

## Part 01

### Question 1: Why can't a struct inherit from another struct or class in C#?

Answer: Because a struct is a Value Type designed to be lightweight and allocated directly on the Stack. Inheritance requires additional memory overhead and dynamic dispatch (V-Table) used by Reference Types on the Heap, which contradicts the core purpose and performance goals of a struct.

### Question 2: How do access modifiers impact the scope and visibility of a class member?

Answer: Access modifiers control where a member can be accessed or modified:

**private:** Accessible only within the same class/struct.

**private protected:** Accessible ONLY by derived classes that are within the same assembly.

**protected:** Accessible within the same class and derived (child) classes.

**internal:** Accessible anywhere within the same project/assembly.

**protected internal:** Accessible anywhere within the same assembly OR by derived classes in other assemblies.

**public:** Accessible from anywhere across the entire application.

### Question 3: Why is encapsulation critical in software design?

Answer: Encapsulation protects internal data from unauthorized or invalid modifications by hiding fields (private) and exposing controlled access points (Properties with validation), ensuring data integrity and making the code easier to maintain.

### Question 4: What is constructors in structs?

Answer: A constructor in a struct is a special method called upon instantiation to initialize all fields and properties of the struct with initial values.

### Question 5: How does overriding methods like ToString() improve code readability?

Answer: Instead of returning the default namespace/type name (e.g., Project.Point), overriding ToString() provides a clear, custom string representation of the object's internal data (e.g., (X: 5, Y: 10)), making logging and debugging much easier to read.

### Question 6: How does memory allocation differ for structs and classes in C#?

Answer:

Structs (Value Types): Allocated directly on the Stack, making them faster to allocate and deallocate as soon as they go out of scope.

Classes (Reference Types): The actual object instance is allocated on the Heap, while only the reference (pointer) is stored on the Stack. Heap allocations are managed and cleaned up later by the Garbage Collector.

## **Part 02**

### **Question 1: What is copy constructor?**

Answer: A copy constructor is a specialized constructor that takes an existing object of the same class/struct as a parameter and copies its data members to create a new, identical instance without altering the original one.

### **Question 2: What is Indexer, when used, as business mention cases u have to utilize it?**

Answer:

Definition: An indexer allows instances of a class or struct to be indexed just like arrays using the [] syntax.

Business Use Case: Used when a class encapsulates an internal collection/list of items and you want to provide clean, direct access to elements by an index or key (e.g., accessing a specific employee from a Company object using company[101]).

### **Question 3: Summarize keywords we have learnt last lecture.**

Answer:

struct: Defines a lightweight Value Type stored on the Stack.

class: Defines a Reference Type stored on the Heap.

new: Allocates memory and invokes object constructors.

private: Restricts member visibility to the containing class/struct.

public: Allows unrestricted member access from anywhere.

internal: Limits member visibility to the same assembly/project.

protected: Grants access to the containing class and its derived classes.

override: Modifies/replaces the inherited implementation of a virtual method.

this: Refers to the current instance of the class or struct.

## Part 03 (Bonus)

### Question: Self-study report

Answer: A concise self-study summary documenting the key OOP concepts explored:

Memory Allocation: Analyzed performance differences between Stack-allocated structs and Heap-allocated classes.

Encapsulation & Scope: Applied access modifiers (private, public, internal, protected) and properties to enforce data protection rules.

Object Customization: Examined parameterized constructors, method overloading, and method overriding (such as customizing ToString()).

### (Bonus)

#### Why C# 10 allowed parameterless struct constructors?

Answer:

Prior to C# 10, the compiler restricted custom parameterless constructors to ensure uniform value-type memory initialization. In C# 10 / .NET 6, this was introduced to give developers control over custom field initialization when instantiating structs via `new MyStruct()`.

Business Case:

Used to override standard zero-initialization with meaningful domain defaults (e.g., setting default coordinates like `X=5, Y=5` or initializing internal collection instances instead of leaving them null).

#### Business Case for Private Constructor?

Answer:

A private constructor prevents external code from instantiating the class directly using the `new` keyword.

Key Use Cases:

Singleton Pattern: Ensures that only a single instance of a class exists throughout the application lifetime (e.g., `DatabaseConnection` or `Logger`).

Factory Methods / Static Factory: Forces instantiation through specific static methods that perform validation or execution logic before object creation (e.g., `User.CreateAdmin()`).